

Lab Sheet 3 — TCP Socket Programming

Echo Client-Server

Course	977-337 Network Programming and Simulation
Student	Hadis Rodpradit
Student ID	6630613039
Program	Digital Engineering (International), College of Computing, PSU Phuket
Environment	Ubuntu 24.04.4 LTS (noble) · Python 3.12.3 · venv: netprog-env

This report documents completion of the six lab tasks for Lab Sheet 3 (a threaded, error-handled TCP echo server and client), followed by the verification checklist and reflection answers. Each task is evidenced by terminal screenshots.

Task 1 — Single client echo

```
diz@diz:~/netprog-2026/lab03_tcp_echo$ python3 server.py
Echo server listening on 0.0.0.0:5000
Connected by ('127.0.0.1', 44726)
```

Server side

```
diz@diz:~/netprog-2026/lab03_tcp_echo$ python3 client.py
Connected to server. Type a message and press Enter ('quit' to exit).
> hello
Echoed back: hello
> █
```

Client side

Figure 1 — The server listens on 0.0.0.0:5000 and accepts a client; the client sends “hello” and receives the identical message echoed back.

Task 2 — Three simultaneous clients (threading)

```
diz@diz:~/netprog-2026/lab03_tcp_echo$ python3 server.py
Echo server listening on 0.0.0.0:5000
Connected by ('127.0.0.1', 48982)
Connected by ('127.0.0.1', 55442)
Connected by ('127.0.0.1', 40108)
```

Server — three connections open at once

```
diz@diz:~/netprog-2026/lab03_tcp_echo$ python3 client.py
Connected to server. Type a message and press Enter ('quit' to exit).
> hello
Echoed back: hello
> hello from client 1
Echoed back: hello from client 1
> █
```

```
diz@diz:~/netprog-2026/lab03_tcp_echo$ python3 client.py
Connected to server. Type a message and press Enter ('quit' to exit).
> hello from client 2
Echoed back: hello from client 2
> █
```

```
diz@diz:~/netprog-2026/lab03_tcp_echo$ python3 client.py
Connected to server. Type a message and press Enter ('quit' to exit).
> hello from client 3
Echoed back: hello from client 3
> █
```

Figure 2 — With per-client threads, the server shows three connections open at the same time and all three clients send and receive echoes independently—none is blocked waiting for another.

Task 3 — Abrupt client disconnect (server survives)

```
diz@diz:~/netprog-2026/lab03_tcp_echo$ python3 client.py
Connected to server. Type a message and press Enter ('quit' to exit).
> hello
Echoed back: hello
> hello from client 1
Echoed back: hello from client 1
> testing disconnect
Echoed back: testing disconnect
> ^CTraceback (most recent call last):
  File "/home/diz/netprog-2026/lab03_tcp_echo/client.py", line 28, in <module>
    main()
  File "/home/diz/netprog-2026/lab03_tcp_echo/client.py", line 17, in main
    message = input("> ")
              ^^^^^^^^^^^
KeyboardInterrupt
```

Client — interrupted with Ctrl+C

```
diz@diz:~/netprog-2026/lab03_tcp_echo$ python3 server.py
Echo server listening on 0.0.0.0:5000
Connected by ('127.0.0.1', 48982)
Connected by ('127.0.0.1', 55442)
Connected by ('127.0.0.1', 40108)
Connection with ('127.0.0.1', 48982) closed
```

Server — logs the closed connection and keeps running

Figure 3 — When the client is killed mid-conversation with Ctrl+C, the server logs the closed connection and continues running instead of crashing, ready to serve other clients.

Task 4 — Address already in use

```
diz@diz:~/netprog-2026/lab03_tcp_echo$ python3 server.py
Traceback (most recent call last):
  File "/home/diz/netprog-2026/lab03_tcp_echo/server.py", line 44, in <module>
    main()
  File "/home/diz/netprog-2026/lab03_tcp_echo/server.py", line 26, in main
    server_sock.bind((HOST, PORT))
OSError: [Errno 98] Address already in use
```

Figure 4 — Starting a second server on port 5000 while the first is still running fails at bind() with OSError [Errno 98].

Explanation. The first server already bound port 5000, and the operating system does not allow two processes to bind the same (IP, port) pair at the same time. When the second server calls bind(), the OS refuses it and raises OSError [Errno 98] Address already in use.

Task 5 — Uppercase echo modification

```
diz@diz:~/netprog-2026/lab03_tcp_echo$ python3 server.py
Echo server listening on 0.0.0.0:5000
Connected by ('127.0.0.1', 60224)
```

Server side

```
diz@diz:~/netprog-2026/lab03_tcp_echo$ python3 client.py
Connected to server. Type a message and press Enter ('quit' to exit).
> hello
Echoed back: HELLO
>
```

Client side

Figure 5 — After a one-line change in handle_client (upper-casing the decoded message before echoing), the client sends “hello” and receives “HELLO”.

Task 6 — Why `recv(1024)` may not match one `send()`, and why `sendall()`

TCP is a continuous byte stream, not a sequence of discrete messages: it does not preserve the boundaries of individual `send()` calls. This means one `recv(1024)` may return fewer bytes than a single `send()` transmitted (the data can arrive split across several `recv()` calls), or it may return bytes from several `send()` calls combined into one. It is therefore the application's job to frame or re-assemble messages. For sending, a plain `send()` may transmit only part of a large payload and returns how many bytes it actually sent, leaving the rest unsent; `sendall()` loops internally until every byte has been sent, so it is the safer choice for larger payloads.

Verification Checklist

#	Checkpoint	Status
1	server.py starts without errors and prints "Echo server listening on 0.0.0.0:5000"	Done
2	A single client can send a message and receive the identical message echoed back	Done
3	A second client attempt while an unthreaded server is busy visibly hangs (blocking-I/O limitation)	Done
4	After adding threading, three simultaneous clients can each send/receive independently	Done
5	Ctrl+C on a client does not crash the server	Done
6	Starting a second server on the same port produces an "Address already in use" error	Done
7	Uppercase modification (Task 5) works correctly	Done

Reflection Questions

1. Explain blocking I/O in the context of `accept()` and `recv()`. What exactly is the server thread doing (from the OS's point of view) while it waits?

Both `accept()` and `recv()` are blocking: they do not return until their event happens. While the server thread waits, the OS puts it to sleep and takes it off the CPU—it is not looping or consuming CPU time, just parked in a blocked state. `accept()` stays blocked until a new client connection arrives; `recv()` stays blocked until data arrives on that connection (or the peer closes it). The OS wakes the thread only when the awaited event occurs.

2. Why is threading needed to serve multiple clients with this blocking socket API? What would you observe if you removed the threading from Part E?

Because the calls block, a single-threaded server that is inside one client's `recv()` loop cannot also be in `accept()` or in another client's `recv()` at the same time—it can only serve one client at a time. Giving each accepted connection its own thread lets the main loop return to `accept()` immediately, so many clients are handled concurrently. If threading were removed, a second client could connect but its messages would hang with no echo until the first client disconnected and freed the server—exactly the limitation demonstrated in Part D.

3. Name a real-world application-layer protocol built on TCP similar to what you built today, and describe one similarity and one difference from your echo protocol.

HTTP is a good example. Similarity: like my echo protocol, it is a connection-oriented client-server protocol over TCP in which the client sends a request and the server responds. Difference: HTTP is a structured protocol with a defined message format—methods, headers, status codes, and body framing (e.g. Content-Length or chunked encoding)—whereas my echo server imposes no format at all and simply sends back whatever raw bytes it receives.